



ELSEVIER

Available online at www.sciencedirect.com

SCIENCE @ DIRECT®

Journal of Computational and Applied Mathematics 196 (2006) 20–44

JOURNAL OF
COMPUTATIONAL AND
APPLIED MATHEMATICS

www.elsevier.com/locate/cam

Interpolation of Lipschitz functions

Gleb Beliakov*

School of Information Technology, Deakin University, 221 Burwood Hwy, Burwood, 3125, Australia

Received 10 March 2005; received in revised form 8 August 2005

Abstract

This paper describes a new computational approach to multivariate scattered data interpolation. It is assumed that the data is generated by a Lipschitz continuous function f . The proposed approach uses the central interpolation scheme, which produces an optimal interpolant in the worst case scenario. It provides best uniform error bounds on f , and thus translates into reliable learning of f . This paper develops a computationally efficient algorithm for evaluating the interpolant in the multivariate case. We compare the proposed method with the radial basis functions and natural neighbor interpolation, provide the details of the algorithm and illustrate it on numerical experiments. The efficiency of this method surpasses alternative interpolation methods for scattered data.

© 2005 Elsevier B.V. All rights reserved.

MSC: 65D05; 41A05; 41A30; 41A50; 41A63

Keywords: Scattered data interpolation; Lipschitz approximation; Optimal interpolation; Central algorithm; Multivariate approximation

1. Introduction

Multivariate data interpolation and approximation is a very common problem in many branches of science. There is a great number of techniques developed for various instances of this problem, such as polynomial regression, spline interpolation and smoothing, wavelets, nearest neighbor search, Sibson interpolation, multivariate adaptive regression splines (MARS), machine learning techniques (e.g., decision trees), neural networks, radial basis functions, etc. For an overview the reader is referred to [1,11,19].

* Tel.: +61 3 9560 9854; fax: +61 3 9251 7604.

E-mail address: gleb@deakin.edu.au.

Shape preserving approximation refers to the approximation problem in which in addition to the data, other information about the function in question is available. For instance, it may be known a priori that the function must be monotone, convex, positive, symmetric, etc. These conditions determine additional constraints on the approximant, which may find explicit representation in terms of the parameters that are fitted to the data. In spline approximation, this problem has been thoroughly studied (see [16,20,21,5]), and such constraints as monotonicity or convexity usually translate into restrictions on spline coefficients.

More recently, the concept of shape preserving interpolation and approximation has been extended to include other known a priori restrictions on the approximant, such as generalized convexity, unimodality, possessing peaks or discontinuities, Lipschitz property, associativity [20,7]. These restrictions require new problem formulations leading to new specific methods of approximation.

In this paper, we consider interpolation of scattered multivariate data, under the assumption that the function that produced these data is Lipschitz continuous. Lipschitz condition implies strict bounds on the difference between the interpolant and the function it models in the Chebyshev max-norm, so that Lipschitz interpolation *guarantees* the performance of the interpolant in the *worst case* scenario, whereas other methods target the average performance. In this sense, Lipschitz approximation translates into *reliable* learning of functions [12,41].

Suppose we have an algorithm A , which allows us compute an approximation to $f(x)$. Such algorithm will produce an error $E(A)$, which is no smaller than the intrinsic error of the problem, $E_{\text{int}} = \inf_A E(A)$. We concentrate on the problem of optimal interpolation, which is to determine an optimal algorithm whose error is precisely E_{int} . We will employ a well-known central interpolatory algorithm, which is always optimal [39].

In the univariate case the optimal central algorithm for Lipschitz functions was studied in [17,35,41]. General formulae in the multivariate case were provided in [35], however they are not computationally efficient, especially when the number of data is large. In this paper, we develop a particularly fast algorithm for computing central optimal interpolant, which uses an alternative representation of the upper and lower bounds via the list of their local optima. We provide explicit formulae for these optima, and also an algorithm for enumerating them.

The resulting optimal interpolant is a piecewise linear function, with a number of desirable features, such as local behaviour of the interpolant, ability to model functions of virtually any shape, simplicity, speed and stability of the evaluation [1,16]. We also obtain continuous dependence of the interpolant on the data, which is *not* the case for many other schemata, such as splines on triangulations [1].

The next section provides the relevant background and the main notions for the rest of the paper. In Section 3 we state the formulae for the central optimal algorithm, which is the basis of our scheme. Section 4 is the main contribution of this paper. In Sections 4.1 and 4.2 we discuss an alternative representation of the interpolant using the list of its local optima, which will help design a particularly fast algorithm for its computation. Section 4.3 presents such an algorithm, which makes computation of the interpolant practical for very large data sets. In Section 5 we discuss the properties of the proposed interpolant and compare it with several existing methods. Finally, Section 6 illustrates the numerical performance of the presented approach using a number of test interpolation problems.

2. Preliminaries

Throughout this paper m will denote the dimensionality of the space, and K will denote the number of data points. Assume that we are given a data set $\mathcal{D} = \{(x^k, y^k)\}_{k=1}^K$, $x^k \in R^m$, $y^k \in R$. We also assume

that y^k are the values of some function $f(x^k) = y^k$, which is unknown to us and which we want to approximate with g , $g \approx f$. Thus we look for an interpolant $g : R^m \rightarrow R$, such that

$$g(x^k) = y^k, \quad k = 1, \dots, K.$$

It is known (e.g., see [18]) that it is impossible to give finite bounds on the values $f(x)$, $x \neq x^k$, $k = 1, \dots, K$ in terms of the data set, if the only additional information is that f is the element of a functional space $\mathcal{V}$, no matter how restricted the space $\mathcal{V}$ is in terms of conditions of continuity, smoothness, analyticity, etc. Therefore, it is meaningless to speak about the goodness of approximation without a reference to some nonlinear constraint on $\mathcal{V}$.

We shall work in the space of continuous functions with the supremum norm, i.e., $\mathcal{V} = C(X)$, $X \subset R^m$, X is compact. We shall assume that f is bounded and Lipschitz continuous,

$$\exists M > 0 : \forall x, z \in X \subset R^m, \quad |f(x) - f(z)| \leq M \|x - z\|, \quad (1)$$

the smallest such number M is called the Lipschitz constant of f in the norm $\|\cdot\|$ and is denoted by $L(f)$. We denote the class of functions whose Lipschitz constant is smaller or equal to M by $\text{Lip}(M)$, i.e.,

$$\text{Lip}(M) = \{f \in C(X) : L(f) \leq M\}.$$

We also assume that the convex hull of the abscissae of data points $CH(\{x^k\}) \subset X$.

Our goal is to find an interpolant g which approximates f well at the points x distinct from the data, given that $f \in \text{Lip}(M)$. We are interested in reliable approximation of f , which means that we want to obtain a good approximation regardless of how inconvenient f is, even in the worst case scenario. That is, we solve the following problem.

Problem 1. Given the data set $\mathcal{D}$, find the optimal interpolating function $g_M : R^m \rightarrow R$,

$$g_M = \arg \inf_{g \in C(X)} \max_{f \in \text{Lip}(M)} \|f - g\|_{C(X)}, \quad (2)$$

such that

$$g(x^k) = f(x^k) = y^k, \quad k = 1, \dots, K.$$

The problem of optimal interpolation was studied in 1970–1980s by Micchelli, Rivlin and Winograd [23–25], Gaffney and Powell [17], Traub and Wozniakowski [39], De Boor [13], Sukharev [35,36], and others. For a detailed introduction to the theory of optimal recovery we refer the reader to the surveys [23,24] and the books [39,37]. Briefly, let A be an algorithm, which allows us compute an approximation to $f(x)$ from some class $\mathcal{S}$. Such an algorithm will always produce an error $E(A)$, which is no smaller than the intrinsic error of the problem, $E_{\text{int}} = \inf_A E(A)$. An optimal algorithm will yield the error E_{int} , i.e., the smallest possible error in the worst case scenario. This smallest possible error is not less than the radius of information for this problem [39].

Golomb and Weinberger [18] have considered the problem of approximation in linear spaces $\mathcal{V}$ subject to finite bounds on some nonlinear functional in a very general setting. They have shown that if the unknown function f is known to lie in a bounded subset $\mathcal{S}$, defined by means of a nonnegative nonlinear positively homogeneous and continuous functional $\rho(u) : \mathcal{S} = \{u \in \mathcal{V} : \rho(u) \leq r\}$, then the range of values of some linear functional $F(f)$ is a closed interval. The optimal approximation to $F(f)$ is the midpoint of this interval, while the error bounds are easily computed as its half-length.

This interpolation scheme is called central, and it is always optimal [39]. In the case of Lipschitz functions, the role of the functional $\rho(u)$ is played by the Lipschitz seminorm $L(f)$ (i.e., $\mathcal{S} = \text{Lip}(M)$). Smolyak lemma [3,39] establishes that there is also an optimal linear algorithm, provided the set $\mathcal{S}$ is balanced, however we shall not use this result and concentrate on the central scheme. Central algorithms possess an even stronger optimality property: they produce the best possible approximation for every fixed $f \in \mathcal{S}$ [39,31].

Central algorithms have been used for a number of different problems, including numerical quadrature [32,37], solution of nonlinear equation and calculation of the topological degree [31], see also [39]. For the univariate interpolation problem, the central scheme was used by Gaffney and Powell [17], Sukharev [35,37], and also in [12,41]. It was shown that optimal interpolation of some classes of functions yields perfect splines [25,13]. We are unaware of multivariate extensions of this result.

Let us recall some other definitions.

Definition 1 (*Minkowski gauge, Rubinov [28], Demyanov and Rubinov [14]*). Let P be a closed star-shaped set in R^m that includes the origin in its interior, i.e., P is closed and if $x \in P$ and $\lambda \in [0, 1]$, then $\lambda x \in P$. Minkowski gauge is the function

$$\mu_P(x) = \inf\{\lambda > 0 : x \in \lambda P\}. \quad (3)$$

It enjoys several interesting properties, as reported in [28,14], in particular it is defined on R^m , is nonnegative and positively homogeneous of degree one. If P is convex, then $\mu_P(x)$ is also convex and sublinear. That is, for convex P we have [28,14]

- (1) $\mu_P(\lambda x) = \lambda \mu_P(x) \quad \forall x \in R^m \quad \forall \lambda > 0$;
- (2) $\mu_P(x + z) \leq \mu_P(x) + \mu_P(z) \quad \forall x, z \in R^m$;
- (3) $\exists c_1, c_2 : c_1 \|x\| \leq \mu_P(x) \leq c_2 \|x\| \quad \forall x \in R^m$.

The above properties suggest that Minkowski gauge can be used as a distance function $d_P(x, z) = \mu_P(x - z)$. Such distance will satisfy all properties of metrics except commutativity ($d_P(x, z) \neq d_P(z, x)$). If P is symmetric with respect to the origin, $d_P(x, z)$ becomes a metric. The triangular inequality for d_P holds, if and only if P is convex [22].

Property (3) of Minkowski gauges ensures that d_P are equivalent to norms. Thus we can write Lipschitz inequality (1) as

$$\exists M > 0 : \forall x, z \in X \subset R^m, \quad |f(x) - f(z)| \leq M d_P(x, z), \quad (4)$$

and the use of d_P instead of the norm will only affect the numerical value of the Lipschitz constant of f . We reiterate that generally d_P is not a metric.

Definition 2 (*Polyhedral distance*). Let P be a convex polytope in R^m , defined by the intersection of r halfspaces, containing the origin in its interior [14, Example 7.2]

$$P = \bigcap_{i=1}^r \{x : x \cdot h_i \leq 1\}, \quad (5)$$

where $h_i \in R^m$ are the directional vectors. The polyhedral distance is

$$d_P(x, y) = \max\{(x - y) \cdot h_i : 1 \leq i \leq r\}.$$

As a special case consider the distance defined by a simplex centered at 0.

Definition 3 (*Simplicial distance*). Let P be a simplex defined as the intersection of $m + 1$ halfspaces (5), defined by vectors

$$\begin{aligned} h_1 &= (-v_1, 0, 0, \dots), \\ h_2 &= (0, -v_2, 0, \dots), \\ &\vdots \\ h_{m+1} &= (v_{m+1}, \dots, v_{m+1}), \end{aligned} \tag{6}$$

$v_i > 0$. The simplicial distance is

$$d_P(x, y) = \max \left\{ \max_{i=1, \dots, m} v_i(y_i - x_i), v_{m+1} \sum_{i=1}^m (x_i - y_i) \right\}. \tag{7}$$

For the purposes of convenience introduce a slack variable $x_{m+1} = 1 - \sum_{i=1}^m x_i$. With the help of the new coordinate, and using $\sum_{i=1}^m (x_i - y_i) = 1 - \sum_{i=1}^m y_i - (1 - \sum_{i=1}^m x_i) = y_{m+1} - x_{m+1}$, we can write (7) in a more symmetric form

$$d_P(x, y) = \max_{i=1, \dots, m+1} v_i(y_i - x_i). \tag{8}$$

Consider a set of points $\{x^k\}_{k=1}^K$, $x^k \in R^m$, called *sites*.

Definition 4. The set

$$\text{Vor}(x^k) = \{x \in R^m : \|x - x^k\| \leq \|x - x^j\|, \forall j \neq k\}$$

is called the Voronoi cell of x^k .

One can choose any norm, or in fact any distance function d_P in this definition. The collection of Voronoi cells for all sites x^k , $k = 1, \dots, K$ is called the Voronoi diagram of the data set. Voronoi diagram is one of the most fundamental data structures of a data set with a long history [2,26,10].

There are multiple extensions of the Voronoi diagram, notably those based on generalization of the distance function [26,10]. One such generalization is called *additively weighted Voronoi diagram*, in which case each site has an associated weight w_k .

Definition 5. Let $\{x^k\}_{k=1}^K$, $x^k \in R^m$ be the set of sites, and $w \in R^K$ be the vector of weights. The set

$$\text{Vor}(x^k, w) = \{x \in R^n : w_k + \|x - x^k\| \leq w_j + \|x - x^j\|, \forall j \neq k\}.$$

is called additively weighted Voronoi cell. The collection of such cells is called additively weighted Voronoi diagram.

3. Optimal Lipschitz interpolation

In this section, we construct an optimal central interpolant to functions from $\text{Lip}(M)$. We need to identify tight upper and lower bounds on the values of f , and then take their half-sum as the solution. From the Lipschitz condition (4) it follows that

$$\forall x, x^k \in X : |f(x) - y^k| \leq M d_P(x, x^k),$$

which yields

$$\max_k (y^k - M d_P(x, x^k)) \leq f(x) \leq \min_k (y^k + M d_P(x, x^k)).$$

Denote the upper and lower bounds by

$$\begin{aligned} H^{\text{upper}}(x) &= \min_k (y^k + M d_P(x, x^k)), \\ H^{\text{lower}}(x) &= \max_k (y^k - M d_P(x, x^k)). \end{aligned} \quad (9)$$

The optimal central interpolant is then given by [17,35,37,39]

$$g(x) = \frac{1}{2} (H^{\text{lower}}(x) + H^{\text{upper}}(x)) \quad \forall x \in X.$$

Note that $H^{\text{upper}}, H^{\text{lower}} \in \text{Lip}(M)$. This follows directly from the fact that function $d_P(\cdot, x^k) \in \text{Lip}(1)$. Consequently $g \in \text{Lip}(M)$ [35]. Also notice that both bounds are tight, which means that there are functions $f \in \text{Lip}(M)$ that interpolate the data, such that $f(x) = H^{\text{upper}}(x)$ or $f(x) = H^{\text{lower}}(x)$ at any x . For instance both $H^{\text{upper}}, H^{\text{lower}} \in \text{Lip}(M)$ and interpolate the data.

Since $\min_k \{a_k\} - \max_j \{b_j\} = \min_k \min_j \{a_k - b_j\}$, and

$$\begin{aligned} H^{\text{upper}}(x) - H^{\text{lower}}(x) &= \min_k \min_j \{y^k + M d_P(x, x^k) - (y^j - M d_P(x, x^j))\} \\ &\leq \min_k \{y^k + M d_P(x, x^k) - (y^k - M d_P(x, x^k))\} \\ &= 2M \min_k d_P(x, x^k). \end{aligned}$$

Thus the maximal error of interpolation, which is the intrinsic error of the problem, is given as

$$\max_{f \in \text{Lip}(M)} \|f - g\|_{C(X)} \leq M \max_{x \in X} \min_{k=1, \dots, K} d_P(x, x^k).$$

The upper and lower bounds are illustrated on Figs. 1 and 2 in the univariate case. They have well-known applications in several fields, such as optimization (e.g., Pijavski–Shubert method [27,29,33]), [34], numerical quadrature [32,39,37] and generation of nonuniform random variates [15, p. 348]. Sikorski [31] uses this technique for solving nonlinear equations and solving the topological degree problem.

The above construction assumes that the information about the Lipschitz constant is correct, i.e., the assumption $f \in \text{Lip}(M)$ is consistent with the data. Otherwise the solution to Problem 1 does not exist. In Section 5.3 we discuss the existence of solution in some detail.

We can use any distance function d_P to build upper and lower approximations and define g , in particular any norm or simplicial distance function. Our aim is to develop a particularly fast algorithm for computing the values of the interpolant, and in the following sections we will employ the simplicial distance.

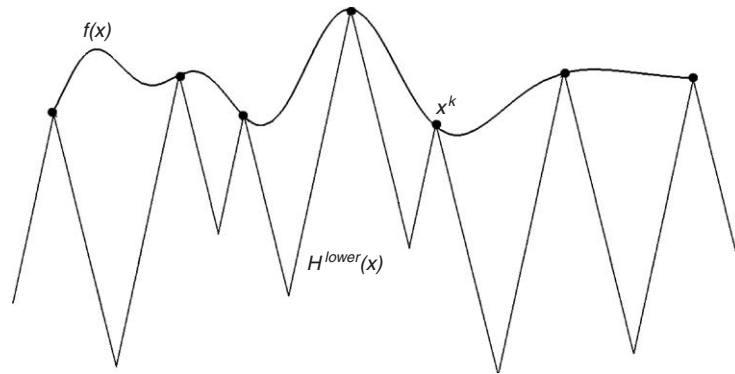
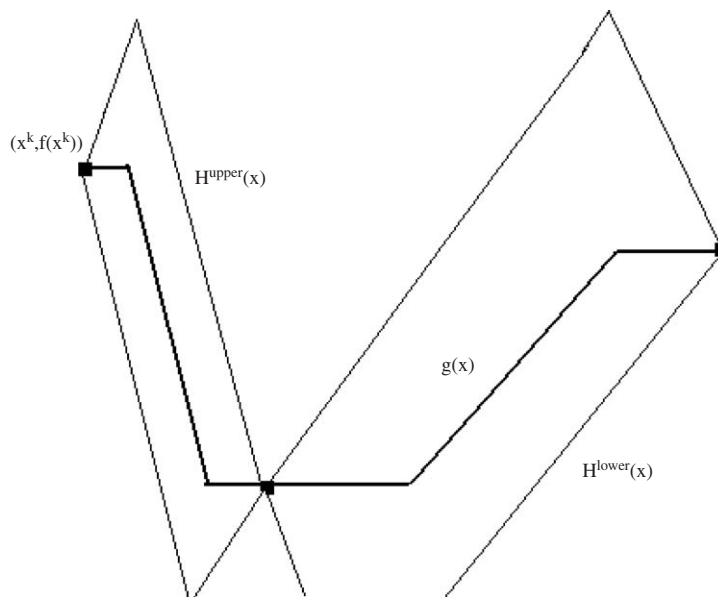


Fig. 1. The saw-tooth underestimate of a Lipschitz function.

Fig. 2. Upper and lower approximations and the interpolant g .

4. Construction of the interpolant

4.1. Building upper and lower saw-tooth estimates

While expressions $H^{\text{upper}}(x)$ and $H^{\text{lower}}(x)$ in (9) provide a way to compute $g(x)$ for every $x \in X$, it may be inefficient as it involves computing pointwise maximum/minimum of K functions, and K could be very large in applications. In this section we present an algorithm for representing $H^{\text{upper}}(x)$ and $H^{\text{lower}}(x)$ through the list of their local maxima and minima, which leads to an efficient evaluation algorithm.

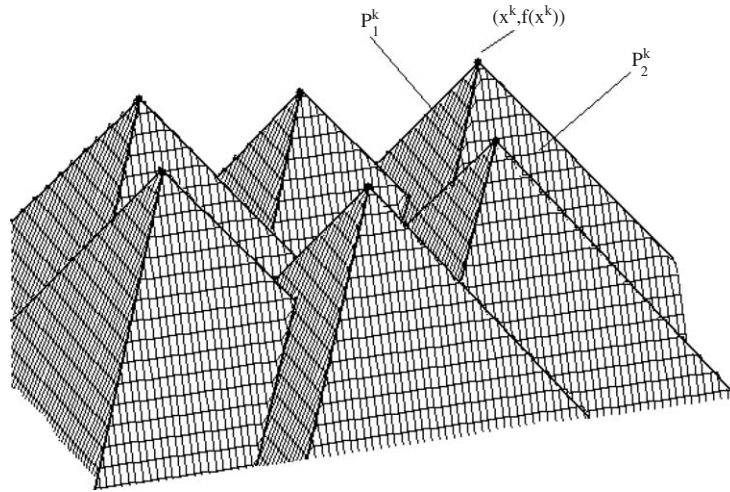


Fig. 3. The lower approximation in case of two variables.

We restrict our attention to the simplicial distance function d_P , defined by (8)

$$d_P(x, y) = \max_{i=1, \dots, m+1} v_i(y_i - x_i),$$

where x_{m+1} denotes the slack variable $x_{m+1} = 1 - \sum_{i=1}^m x_i$. In other words we consider the hyperplane

$$HP = \left\{ x \in R^n : \sum_{i=1}^n x_i = 1 \right\},$$

with $n = m + 1$, which coincides with our original space R^m . In this case the bounds are given by

$$\begin{aligned} H^{\text{lower}}(x) &= \max_{k=1, \dots, K} \min_{i=1, \dots, n} \left(y^k - C_i (x_i^k - x_i) \right), \\ H^{\text{upper}}(x) &= \min_{k=1, \dots, K} \max_{i=1, \dots, n} \left(y^k + C_i (x_i^k - x_i) \right), \quad x \in HP, \end{aligned} \quad (10)$$

where $C_i = M v_i$, and

$$\max_f \|f - g\|_{C(X)} = \max_{x \in X} \min_{k=1, \dots, K} \max_{i=1, \dots, n} C_i (x_i^k - x_i).$$

Let us build the lower approximation $H^{\text{lower}}(x)$ using (10). The upper approximation can be obtained from the lower in an obvious way: consider the function $\bar{f} = -f$; find its lower approximation $\bar{H}^{\text{lower}}$; then set $H^{\text{upper}} = -\bar{H}^{\text{lower}}$.

The graph of the function H^{lower} is illustrated on Figs. 1 and 3. It is a piecewise linear function, whose maxima are attained at points x^k . Next, we will identify its local minima in R^m . Of course, H^{lower} is unbounded from below, but it has a number of finite local minima. We will show that each local minimizer x^* of H^{lower} has an associated subset $A(x^*)$ on which it is unique, and that H^{lower} has a simple expression

on $A(x^*)$, and hence it is easily computable. Our method of computation of g will rely on the ability to identify the subset $A(x^*)$ to which x belongs, thus avoiding comparison of K values in (10).

Let us introduce the *support vectors* $l^k \in R^n$, $k = 1, \dots, K$:

$$l_i^k = \frac{y^k}{C_i} - x_i^k. \quad (11)$$

We are interested in the local minima of the function

$$H^{\text{lower}}(x) = \max_{k=1,\dots,K} h^k(x) = \max_{k=1,\dots,K} \min_{i=1,\dots,n} C_i(l_i^k + x_i), \quad (12)$$

where functions $h^k(x) = \min_{i=1,\dots,n} C_i(l_i^k + x_i)$ are called the support functions. We have the following:

Theorem 1 (Beliaikov [8]). *The necessary and sufficient condition for a point x^* to be a local minimizer of $H^{\text{lower}}(x)$ given by (12) is that there exist an index set $J = \{k_1, k_2, \dots, k_n\}$ of cardinality n , such that*

$$d = H^{\text{lower}}(x^*) = C_1(l_1^{k_1} + x_1^*) = C_2(l_2^{k_2} + x_2^*) = \dots = C_n(l_n^{k_n} + x_n^*),$$

and $\forall i \in \{1, \dots, n\}$,

$$C_i(l_i^{k_i} + x_i^*) < C_j(l_j^{k_i} + x_j^*), \quad j \neq i.$$

Theorem 1 has a direct geometrical interpretation. Consider the hypographs of functions h^k (hyp $h^k = \{(x, y) : y \leq h^k(x)\}$). These hypographs are unbounded polyhedrons with the unique vertex at (x^k, y^k) , bounded by n planes intersecting at (x^k, y^k) (Fig. 3). The slope of each plane in the direction of x_i is C_i . Let us denote the i th facets of these polyhedrons by P_i^k , $P_i^k = \{(x, y) : x \in R^n, \sum x_i = 1, y = h^k(x) = C_i(l_i^k + x_i)\}$. Notice that the i th facets of two different hyp h^r and hyp h^q , P_i^r and P_i^q , are either parallel or belong to the same plane.

Let x^* be a local minimizer of $H^K(x)$, which corresponds to some index set J satisfying the conditions of the theorem. Form the ordered combination of the support vectors $L = \{l^{k_1}, l^{k_2}, \dots, l^{k_n}\}$ that corresponds to J . It is helpful to visualize this combination as a matrix L whose rows are the support vectors l^{k_i} :

$$L = \begin{pmatrix} l_1^{k_1} & l_2^{k_1} & \dots & l_n^{k_1} \\ l_1^{k_2} & l_2^{k_2} & \dots & l_n^{k_2} \\ \vdots & \vdots & \ddots & \vdots \\ l_1^{k_n} & l_2^{k_n} & \dots & l_n^{k_n} \end{pmatrix}, \quad (13)$$

so that its components are given by $L_{ij} = (y^{k_i} / C_j) - x_j^{k_i}$.

Theorem 2 (Beliaikov [8]). *Let the support vectors l^k , $k = 1, \dots, K$ be defined using (11). Let x^* denote a local minimizer of $H^{\text{lower}}(x)$ and $d = H^{\text{lower}}(x^*)$. Then matrix (13) corresponding to x^* enjoys the following properties:*

$$(1) \quad \forall i, j \in \{1, \dots, n\}, i \neq j : l_i^{k_j} > l_i^{k_i};$$

- (2) $\forall r \notin \{k_1, k_2, \dots, k_n\} \exists i \in \{1, \dots, n\} : L_{ii} = l_i^{k_i} \geq l_i^r$;
 (3) $d = (\text{Trace}(L) + 1)/C^-$, and
 (4) $x_i^* = (d/C_i) - l_i^{k_i}$,

where $C^- = \sum_{i \in I} (1/C_i)$.

Conditions (1) and (2) of the Theorem 2 are easily interpreted. Condition (1) implies that the diagonal elements of matrix L are *dominated* by their respective columns, and condition (2) implies that no support vector l^r (which is not part of L) strictly dominates the diagonal of L . We shall use Theorem 2 to convert the problem of locating all local minimizers of H^{lower} to the combinatorial problem of enumerating all combinations L with the above mentioned properties (1) and (2). Once we enumerated all such combinations, the positions of local minima x^* and their values d are easily computed from (3) and (4).

Consider a local minimizer x^* of H^{lower} . According to Theorem 2, we have a combination of support vectors L which satisfies the four conditions of the theorem. Let us assume for the moment that the values y^{k_i} are unknown to us, and try to determine the set $A(L)$ where x^* can be located. From conditions (1) and (4) we have for every $i, j \in \{1, \dots, n\}, i \neq j$

$$(x_j^* + l_j^{k_j})C_j = d = (x_i^* + l_i^{k_i})C_i < (x_i^* + l_i^{k_j})C_i.$$

Replacing $l_i^{k_j}$ with (11)

$$C_j x_j^* + f(x^{k_j}) - C_j x_j^{k_j} < C_i x_i^* + f(x^{k_j}) - C_i x_i^{k_j},$$

and hence

$$\forall i, j \in \{1, \dots, n\}, \quad i \neq j : C_j(x_j^* - x_j^{k_j}) < C_i(x_i^* - x_i^{k_j}). \quad (14)$$

The system of $n \times (n-1)$ inequalities (14) defines the set $A(L)$ of possible locations of the local minimizer x^* through the points x^{k_i} . It defines the set of points on which

$$H^{\text{lower}}(x) = \max_{i=1, \dots, n} C_i(x_i + l_i^{k_i}). \quad (15)$$

Clearly, there is only one local minimizer of H^{lower} on $A(L)$, namely x^* . This set is illustrated in Fig. 4. The collection of the closures of the sets $\tilde{A}(L)$ forms a partition of A .

It is also interesting to consider another partition of A generated by the function H^{lower} in (12). Consider the sets of points on which H^{lower} coincides with one of the support functions h^k :

$$S^k = \{x \in R^m : Md_P(x, x^k) - y^k = h^k(x) \geq h^j(x) = Md_P(x, x^j) - y^j, \forall j \neq k\}.$$

We observe that the sets S^k are precisely the additively weighted Voronoi cells $\text{Vor}(x^k, w)$ (see Section 2), with the weights given by $w_k = -(y^k/M)$. Thus H^{lower} generates an additively weighted Voronoi diagram, and the lower approximation itself can be written as

$$H^{\text{lower}}(x) = h^k(x) \quad \text{if } x \in \text{Vor}(x^k, w).$$

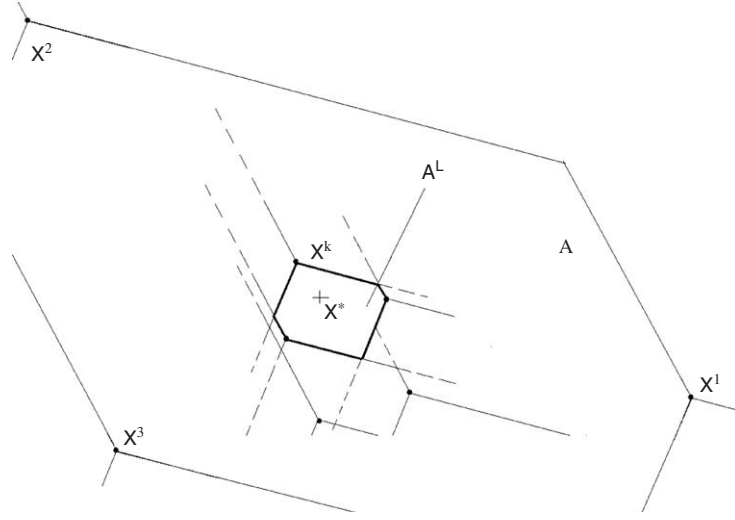


Fig. 4. The sets $A(L)$ on which each local minimizer is unique.

4.2. Enumeration of local minimizers of H^{lower}

The naive approach to enumerating local minimizers of H^{lower} is to try out all possible combinations of support vectors L and leave the ones that satisfy conditions (1) and (2). The computational complexity of this exercise is $O\left(\binom{K}{n}\right)$, which is unacceptably large. The combinations L can also be built incrementally, by adding one new support vector at a time.

Let us assume that the data set has the first $n=m+1$ data points with the i th coordinate $x_i^i = P \rightarrow \infty$, $i = 1, \dots, n$, and with all the other coordinates and y^i are finite. If this is not the case, we can always add n auxiliary data points at the beginning of the data set, and consider the augmented data set which would satisfy this assumption. It is quite clear that

$$H^n(x) = \max_{i=1, \dots, n} h^i(x) = \max_{i=1, \dots, n} C_i(l_i^i + x_i) < H^{\text{lower}}(x) \quad \forall x \in HP,$$

and hence the auxiliary data points do not participate in calculating the value of H^{lower} . On the other hand, the unique local minimum of H^n is easily identified from Theorem 2 using combination $L = \{l^1, l^2, \dots, l^n\}$.

Denote by H^k the lower approximation (12) built using the first k data points. Clearly,

$$H^k(x) = \max\{H^{k-1}(x), h^k(x)\}, \quad k = 1, \dots, K. \quad (16)$$

Suppose, we already have identified all local minima of $H^{k-1}(x)$, i.e., all the required combinations L . Let us denote this set by V^{k-1} . When we add another support vector l^k , and $x^k \in A$, then we can “inherit” most of the local minima of $H^{k-1}(x)$, and we only need to add a few new local minima, that are new combinations L necessarily involving l^k . We proved in [4] that these new combinations are simple modifications of those combinations that were discarded because they failed (2) with $l^r = l^k$.

To put it simply, the local minima of $H^k(x)$ in (16) are either the same as those of $H^{k-1}(x)$, or appear in place of those that disappear when $h^k(x)$ is added. The new minima will necessarily be formed by one of the facets of $\text{hyp } h^k$. Our assumption about the first n data points implies that $A = HP$, therefore all $x^k \in A$, $k = 1, \dots, K$ as needed.

Furthermore, in [6,8] we proved that all local minimizers of functions $H^n, H^{n+1}, \dots, H^k, \dots, H^K$ can be seen as nodes of a tree, and the minimizers of H^K are the leaves of this tree. The root of the tree is $\{l^1, l^2, \dots, l^n\}$. The parent and child nodes of this tree differ only by one support vector, and if a child node fails test (2), its parent (and all ancestors) will also fail it. It is possible to perform the test of condition (2) by starting the test from the root and performing it on all the nodes (depth-first search). The computational complexity of test (2) is then $O(\log |V^K|)$, where $|V^K|$ is the number of local minima of $H^K(x)$ (provided the tree is balanced).

We reproduce the recursive algorithm of updating the tree of local minima T^{K-1} to T^K from [6,8].

Algorithm 1 (update of the tree T^{K-1}).

Purpose: Given the representation of all local minimizers of functions $H^n, H^{n+1}, \dots, H^{K-1}$ as nodes of a tree, and a new support function h^K , update the tree structure to include local minimizers of H^K .

Input: The tree T^{K-1} containing local minimizers of $H^n, H^{n+1}, \dots, H^{K-1}$; the new support vector l^K ; tested node L .

Output: The tree T^K .

Note: The trees T^{K-1} and T^K share the same memory and are accessed through the root. Assignment at Step 5 is symbolic.

Step 1: Test L against condition (2) of Theorem 2, with $l^r = l^K$.

Step 2: If test succeeds, go to Step 5 (cut off this branch).

Step 3: If test fails, and L is not a leaf, then call Algorithm 1 ($T^{K-1}, l^K, \text{child}(L), T^K$) for all children of L . Go to Step 5.

Step 4: Otherwise (test failed, and L is a leaf) add n children to L . Each child node is a copy of L , with l^{k_i} replaced with l^K in the i th child. Test condition (1) of Theorem 2 for each child. If test fails, delete this child node.

Step 5: If L is V^n (root), then $T^K = T^{K-1}$.

Return.

It is worth noting that computation of d in Theorem 2 can be simplified using parent–child relations between the nodes of the tree. The parent node is $L_p = \{l^{k_1}, \dots, l^p, \dots, l^{k_n}\}$ and the child node is $L_c = \{l^{k_1}, \dots, l^c, \dots, l^{k_n}\}$ differ by one support vector (l^p vs. l^c) in the i th position. Then

$$d(L_c) = \frac{\text{Trace}(L_c) + 1}{C^-} = \frac{\text{Trace}(L_p) - l_i^p + l_i^c + 1}{C^-} = d(L_p) + \frac{l_i^c - l_i^p}{C^-}. \quad (17)$$

The test of condition (2) at step 1 can be done in $O(1)$ operations for all the nodes except the root (for which it always fails). Indeed, we test the node L only if its parent has failed the test. Then we only need to compare the i th elements of the vectors l^r and l^{k_i} .

Below is the algorithm for enumerating local minima of H^{lower} . The algorithm starts with an empty tree T^K , and grows this tree by adding one support vector at a time.

Algorithm 2 (Construction of the tree T^K).

Purpose: Given the dataset $D = \{(x^k, y^k)\}_{k=1}^K$, $x^k \in R^m$, $y^k \in R$, build the tree of local minima of $H^n, H^{n+1}, \dots, H^K = H^{\text{lower}}$.

Input: The dataset D .

Output: The tree T^K .

Note: We use the assumption that $x_k^k \rightarrow \infty$, $k = 1, \dots, n$, which would involve adding n auxiliary data points to the data set D , so that the original data are re-labeled $x^{1+n}, x^{2+n}, \dots, x^{K+n}$.

Step 0: Add auxiliary data to the dataset D . Preprocess the data set to compute the slack variables $x_n^k, k = 1, \dots, K$. Form the basis support vectors $l^1, \dots, l^n$ using (11).

Step 1: Set $\text{root}(T^K) = \{l^1, \dots, l^n\}$.

Step 2: For $k = 1 + n, \dots, K + n$ do:

Step 2.1: Form new support vector l^k using (11).

Step 2.2: Call Algorithm 1($T^K, l^k, \text{root}(T^K), T^K$)

Step 3: Stop.

Thus we have obtained an alternative representation of functions H^{upper} and H^{lower} in (10), by enumerating their local minima and maxima. The Algorithm 2 is applied twice, first to represent H^{lower} , and then to represent H^{upper} using a modified data set in which all y^k have inverted sign. As the output, we obtain two trees, T^{lower} and T^{upper} . Let us consider how H^{lower} and H^{upper} can be evaluated at an arbitrary $x \in X$.

4.3. Evaluation of $H^{\text{lower}}(x)$

Given $x \in X$, we are interested in computing the value $H^{\text{lower}}(x)$, given by (12),

$$H^{\text{lower}}(x) = \max_{k=1, \dots, K} h^k(x) = \max_{k=1, \dots, K} \min_{i=1, \dots, n} C_i(l_i^k + x_i).$$

We want to avoid comparing K values in the max operator, and to use

$$H^{\text{lower}}(x) = \max_{k \in V} \min_{i=1, \dots, n} C_i(l_i^k + x_i),$$

where V is a greatly reduced subset of $\{1, \dots, K\}$.

To compute the subset V , we will use the existing tree structure T^{lower} and a modification of the Algorithm 1. Suppose that the maximum in (12) is achieved when $k=p$, i.e., $H^{\text{lower}}(x) = \min_{i=1, \dots, n} C_i(l_i^p + x_i)$. Further, let the minimum be reached when $i=j$, that is $H^{\text{lower}}(x) = C_j(l_j^p + x_j)$. Now let us add a new auxiliary datum (x, u) to the data set and build the support function $\tilde{h}(t) = u - d_P(t, x) = \min_{i=1, \dots, n} C_i(\tilde{l}_i + t_i)$, with $\tilde{l}_i = (u/C_i) - x_i$. Next, we will show that for at least one local minimizer x^* of H^{lower} , which involves the support function h^p , $\tilde{h}(x^*) > H^{\text{lower}}(x^*)$. In other words, x^* is no longer a local minimizer of the function $\tilde{H}(t) = \max\{H^{\text{lower}}(t), \tilde{h}(t)\}$.

Notice that Algorithm 1 detects this situation in its test of condition (2) of Theorem 2, and it does it in $O(\log |V^K|)$ operations, where $|V^K|$ is the number of local minimizers of H^{lower} . Thus, we can apply Algorithm 1 to detect all such minimizers x^* for which $\tilde{h}(x^*) > H^{\text{lower}}(x^*)$, and automatically exclude those h^k from (12) which do not participate in forming these minimizers.

Theorem 3. Let A be defined by (14), H^{lower} be given by (12), and let for a given $x \in A$ the maximum and minimum if this expression are reached at $k = p$ and $i = j$, respectively, i.e., $H^{\text{lower}}(x) = C_j(l_j^p + x_j)$. Then there exists a combination of support vectors $L = \{l^{k_1}, \dots, l^{k_n}\}$ which satisfies conditions (1) and (2) of Theorem 2, which involves l^p , and

$$H^{\text{lower}}(x^*) < \tilde{h}(x^*) = u - Md_P(x^*, x),$$

where $u > \max\{f(x^{k_1}), \dots, f(x^{k_n})\}$.

Proof. Let us denote by Q_i^k the following sets

$$Q_i^k = \{x \in HP : h^k(x) = C_i(l_i^k + x_i)\}, \quad k = 1, \dots, K, \quad i = 1, \dots, n.$$

The condition of this theorem translate into $x \in Q_j^p$. We also know that sets $A(L)$ defined by (14) form a partition of A , that is x belongs to the closure $\bar{A}(L)$ of at least one such set. Since on $\bar{A}(L)$ H^{lower} is given by (15)

$$H^{\text{lower}}(x) = \max_{i=1, \dots, n} C_i(l_i^{k_i} + x_i),$$

we conclude that p must be one of the k_i in the above expression, namely $p = k_j$. Thus there exists a local minimizer x^* of H^{lower} , such that $d = H^{\text{lower}}(x^*) = C_j(l_j^p + x_j^*)$.

Suppose that we know the value $f(x) \geq H^{\text{lower}}(x)$. Take $u > f(x)$ and form the support vector $\tilde{l}$ using $\tilde{l}_i = (u/C_i) - x_i$, and the support function $\tilde{h}(t) = \min_{i=1, \dots, n} C_i(\tilde{l}_i + t_i)$.

Denote by $\tilde{Q}_i = \{x \in HP : \tilde{h}(t) = C_i(\tilde{l}_i + t_i)\}$, $i = 1, \dots, n$. x^* belongs to at least one of these sets, let it be $x^* \in \tilde{Q}_i$, that is $\tilde{h}(x^*) = C_i(\tilde{l}_i + x_i^*)$. Since x^* is a local minimizer, by Theorem 1 $H^{\text{lower}}(x^*) = C_i(l_i^{k_i} + x_i^*)$.

On the other hand, $x \in \bar{A}(L)$ implies (15), and hence

$$C_i(\tilde{l}_i + x_i) = \tilde{h}(x) = u > H^{\text{lower}}(x) \geq C_i(l_i^{k_i} + x_i).$$

From the last inequality it follows $\tilde{l}_i > l_i^{k_i}$, and then

$$\tilde{h}(x^*) = C_i(\tilde{l}_i + x_i^*) > C_i(l_i^{k_i} + x_i^*) = H^{\text{lower}}(x^*).$$

Of course $f(x)$ is unknown to us. However on $A(L)$ expression (15) translates into

$$f(x) \geq H^{\text{lower}}(x) = \max_{i=1, \dots, n} (f(x^{k_i}) - Md_P(x, x^{k_i})) \geq \max_{i=1, \dots, n} f(x^{k_i}),$$

and therefore we can choose u as in the condition of this theorem. $\square$

An important implication of Theorem 3 is its converse: if for some local minimizer x^* we have $H^{\text{lower}}(x^*) \geq u - Md_P(x^*, x)$, then $H^{\text{lower}}(x) > C_i(l_i^{k_i} + x_i)$, $i = 1, \dots, n$. We can check all local minimizers, and if we establish that for some l^p we always have $H^{\text{lower}}(x) > C_i(l_i^p + x_i)$, $i = 1, \dots, n$, then

we do not need to take $k = p$ in (12). This way we can reduce the set $\{1, \dots, K\}$ of support functions in (12) that need to be compared.

Notice that the number of local minimizers of H^{lower} is much greater than the number of support functions K , for $m \geq 2$. We already noted that H^{lower} can be represented in terms of an additively weighted Voronoi diagram, whose vertices correspond to the local minima of H^{lower} . The number of the vertices of the Voronoi diagram in the simplicial distance function grows as $O(K^{\lceil m/2 \rceil})$ on average, but can grow as $O(K^m)$ in the worst case [10]. Thus, it looks as if we do a much greater job by testing all local minimizers, whose number is exponential in m , in order to avoid K comparisons in (12).

However the local minimizers are organized into a tree structure, for which the processing of local minimizers takes only logarithmic time of their number. Then we have $\log(K^n) = n \log K$, and therefore the average complexity of the evaluation step will be $O(\lceil m/2 \rceil \log K)$, i.e., logarithmic in K . The exhaustive evaluation using Eq. (9) is linear in K . Thus, our approach indeed reduces the computation time for large K , as seen from Table 3.

5. Discussion

5.1. Properties of the interpolant

Besides providing the best uniform estimate of the unknown function f in the worst case scenario, the proposed method of Lipschitz interpolation has a few useful features.

- (1) The upper bound on the error of approximation is $C = M \max_x \min_k d_P(x, x^k)$. This upper bound provides a guarantee on the quality of approximation regardless the distribution of data points or which particular function $f \in \text{Lip}(M)$ generated these data.
- (2) Preservation of the range of the data: $\min_k \{y^k\} \leq g(x) \leq \max_k \{y^k\}$.
- (3) If $d_P(x, x^k)$ is chosen as the simplicial distance (8) (or any polyhedral distance function), then the interpolant is a piecewise linear function.
- (4) $g \in \text{Lip}(M)$.
- (5) g depends continuously on the data.
- (6) The interpolant g provides a local approximation scheme (i.e., values of g depend only on the nearest data points).

Properties (1) and (4) were proven in [35]. To prove (2), let the maximum and minimum in (9) be reached at $k = p$ and $k = q$, respectively. Then

$$2g(x) = y^p - Md_P(x, x^p) + y^q + Md_P(x, x^q).$$

It immediately follows that

$$2y^q \leq y^p - Md_P(x, x^p) + y^q + Md_P(x, x^q) \leq 2y^p.$$

The remaining properties are obvious. Note that many of these properties are the desirable features of any multivariate interpolant, as listed in [1].

We would like to mention that the error bounds (in property (1) above) cannot be improved by adding conditions of differentiability or analyticity of the unknown function f . This directly follows from the

fact that the worst case bounds H^{lower} and H^{upper} can be approximated uniformly and with arbitrary precision by analytic functions (e.g., polynomials). Thus there are functions $f \in \text{Lip}(M) \cap C^\infty(X)$, such that $f(x^k) = y^k$ and $|f(x) - H^{\text{lower}}(x)| \leq \varepsilon$ (or $|f(x) - H^{\text{upper}}(x)| \leq \varepsilon$) for any given $\varepsilon > 0$ and $x \in X$.

5.2. Parallels with other interpolation schemata

5.2.1. Radial basis functions

It is useful to look at parallels of the proposed approach with some existing methods. First consider a popular method of radial basis functions [1,11,19]

$$s(x) = \sum_{k=1}^K \alpha_k \phi(\|x - x^k\|).$$

This scheme is obviously global, but by taking most α_k as zeroes, with the exception of those that correspond to data points close to x , the scheme can be made local. Multiquadrics is a special case of this scheme when $\phi(t) = (a^2 - t^2)^{1/2}$. Sheppard's method is another related scheme based on Euclidean distances from the data points

$$s(x) = \sum_{k=1}^K w_k(x) y^k \quad \text{where } w_k(x) = \frac{\|x - x^k\|^{-p}}{\sum_{k=1}^K \|x - x^k\|^{-p}}, \quad p > 1.$$

Our method can also be seen from the perspective of radial basis functions, but it is different in that instead of linear combinations we take combination of max and min:

$$g(x) = \frac{1}{2} \max_k \phi_1(\|x - x^k\|, y^k) + \frac{1}{2} \min_k \phi_2(\|x - x^k\|, y^k),$$

where $\phi_1(\|x - x^k\|, y^k) = y^k - M\|x - x^k\|$, $\phi_2(\|x - x^k\|, y^k) = y^k + M\|x - x^k\|$. Besides, we also use other distances d_p instead of the Euclidean norm. The locality of our method is ensured automatically. It does not require solving a large linear system ($K \times K$) to determine coefficients α_k .

5.2.2. Natural neighbor interpolation

Another popular method of multivariate interpolation is the natural neighbor scheme by Sibson [30]. The idea of this method is to build an interpolant whose value at x would depend on a few data points close to x at the same time distributed *all around* x . It favorably contrasts with the nearest neighbor methods in which only the distance from x matters.

In the natural neighbor scheme, the interpolant is a weighted average of the neighboring data values

$$s(x) = \sum_{j=1}^J w_j(x) y^j,$$

where the weight $w_j(x)$ is proportional to the volume of the part of Voronoi cell $\text{Vor}(x^j) = \{z : \|z - x^j\| \leq \|z - x^k\|, k \neq j\}$, which is cut by the Voronoi cell $\text{Vor}(x) = \{z : \|z - x\| \leq \|z - x^k\|\}$, when x is added to the Voronoi diagram as one of the sites. Since Voronoi cell $\text{Vor}(x)$ borders only a few neighboring Voronoi cells, only a few neighboring data points *around* x participate in calculation of

$s(x)$ (so called natural neighbors). The scheme is obviously local. More recently, variations of Sibson's method were developed, based on other rules for calculating weights $w_j(x)$ [9].

One of the advantages of such a scheme over methods based on triangulations (e.g., Delaunay triangulation) is continuous dependence of the interpolant on data points. Sibson's interpolant possesses other useful properties, but it is computationally expensive, as each x requires computation of a new Voronoi diagram having x as one of the sites. There are methods that allow an update of the Voronoi diagram when x is added to the list of sites in 2- and 3-variate cases, so that the whole Voronoi diagram needs not be built for every x . Such methods are very competitive, but we are unaware of any extension for more than three variables.

The presented method can also be seen from the point of view of building Voronoi diagrams. However, it involves more complicated weighted Voronoi diagrams. Efficient methods for computation of additively weighted Voronoi diagrams are available only in two variables.

5.2.3. Triangulation based methods

The dual of the Voronoi diagram is Delaunay (pre-)triangulation. It is an optimal triangulation in a certain sense (e.g., it maximizes the minimum angle of the triangles, minimizes the maximum radius of a min-containment sphere [2,26]), and is frequently used in scattered data interpolation. For instance, one can easily build a piecewise linear interpolant (linear spline), using barycentric coordinates.

Because of the duality, constructions of the Voronoi diagram and of Delaunay triangulation are essentially done in the same way, and the complexity of both is exponential in m . Evaluation of the interpolant involves identification of the element of the partition containing x (either Voronoi cell or an element of Delaunay triangulation), and this is quite expensive for more than three variables.

Our approach is based on a related concept of additively weighted Voronoi diagram [26,10]. The set of points

$$\{x \in R^m : h^k(x) = y^k - Md_P(x, x^k) \geq h^j(x), j \neq k\}$$

is precisely the weighted Voronoi cell

$$\text{Vor}(x^k; w) = \{x \in R^m : d_P(x, x^k) + w_k \leq d_P(x, x^j) + w_j, j \neq k\}$$

with respect to the distance d_P , where the weights $w \in R^K$ are found as $w_k = -(y^k/M)$. Thus, the lower approximation can be written as

$$H^{\text{lower}}(x) = h^k(x) \quad \text{if } x \in \text{Vor}(x^k; w).$$

The upper approximation is written analogously, however with different weights $w_k = y^k/M$.

The two weighed Voronoi diagrams (for the lower and upper approximations) do not coincide. If this were the case, we would obtain a piecewise constant interpolant equal to the data values y^k in each $\text{Vor}(x^k)$, which is the original Thiessen's interpolant [38]. This limiting case arises if we take $M \rightarrow \infty$. As in the Sibson's method, the data points that participate in calculating $g(x)$ are natural neighbors, and there are at most two of them for every x .

In terms of the computational complexity, our approach requires building two weighted Voronoi diagrams. The existing methods provide solution only in the bivariate case. Evaluation of the interpolant requires similar tasks as in Sibson's or piecewise linear interpolation, namely efficient identification of the element of the partition that contains x . The use of simplicial distance functions appears to reduce

computational costs. As the following section shows, both construction of the interpolant and its evaluation are competitive in terms of computing time.

5.3. Identification of the Lipschitz constant

Throughout this paper we assumed that the Lipschitz constant of f in a particular distance d_P was provided (e.g., as part of the interpretation of the data). We emphasize that only in this case (i.e., when we have the data and the condition $f \in \text{Lip}(M)$, or some other nonlinear restriction on f , according to [18]) one can obtain finite error bounds and construct an optimal interpolant.

However, a more typical situation is when the only information is the data set $\mathcal{D}$ itself. In this case one has to estimate the value of the Lipschitz constant to adequately choose the class of functions $\text{Lip}(M)$. If the class is too narrow, Problem 1 may not have a solution, whereas if the chosen class is too wide, the intrinsic error of the interpolation problem becomes too large. In this section, we address the issue of estimation of the Lipschitz constant from the data set, and the consequences of its under- and overestimation.

First, consider the compatibility of $\text{Lip}(M)$ with the available data. We call the data set $\mathcal{D}$ compatible with the class $\text{Lip}(M)$, if there exists a function $f \in \text{Lip}(M)$ that $f(x^k) = y^k$, $k = 1, \dots, K$. Obviously, the solution to Problem 1 exists only if the data are compatible with the given Lipschitz condition.

Theorem 4. *The data set $\mathcal{D}$ is compatible with the class $\text{Lip}(M)$ if and only if the following conditions hold*

$$\forall i, j \in \{1, \dots, K\} : |y^i - y^j| \leq M d_P(x^i, x^j) \quad (18)$$

Proof. Assume the above condition holds. Let us define the function

$$H^{\text{lower}}(x) = \max_k (y^k - M d_P(x, x^k)).$$

This function is a pointwise maximum of functions

$$h^k(x) = y^k - M d_P(x, x^k);$$

since $d_P(x, \cdot) \in \text{Lip}(1)$, each $h^k \in \text{Lip}(M)$, and hence $H^{\text{lower}} \in \text{Lip}(M)$. On the other hand,

$$h^k(x^k) = y^k \geq y^j - M d_P(x^j, x^k)$$

(which follows from (18)), and therefore $H^{\text{lower}}(x^k) = h^k(x^k) = y^k$. Hence H^{lower} interpolates the data.

Conversely, assume that for some i, j

$$|y^i - y^j| > M d_P(x^i, x^j).$$

Then any interpolant f will violate the Lipschitz condition implied in $\text{Lip}(M)$

$$|f(x^i) - f(x^j)| > M d_P(x^i, x^j). \quad \square$$

Since $M_1 > M_2$ implies $\text{Lip}(M_2) \subset \text{Lip}(M_1)$, compatibility with the data will not be affected by choosing a larger Lipschitz constant.

Methods of estimation of the Lipschitz constant of functions exist [40]. Our construction itself serves to estimate M precisely in the required distance d_P . Using Theorem 4, we can calculate the smallest Lipschitz constant M_{data} compatible with a given data set as

$$M_{\text{data}} = \min\{C \in R : \forall k, j = 1, \dots, K, |y^k - y^j| \leq C d_P(x^k, x^j)\}. \quad (19)$$

This implies that we choose the smallest class of Lipschitz functions $\text{Lip}(M_{\text{data}})$ still compatible with the data.

6. Numerical experiments

To illustrate the performance of the central interpolant g we approximate the following Lipschitz functions using randomly scattered data points in the indicated domain.

Test function 1

$$f(x) = \sin x_1 \sin x_2 + 0.05(\sin 5x_1 \sin 5x_2)^3, \quad x \in [0, 1]^2.$$

Test function 2

$$f(x) = \sin 5x_1 \sin 2x_2 + 0.2(\sin 20x_1 \sin 20x_2)^3, \quad x \in [0, 3]^2.$$

Test function 3

$$f(x) = \prod_{i=1}^m \sin 2x_i, \quad x \in [0, 3]^m.$$

The approximations of test functions 1 and 2 are plotted on Figs. 5–8. These test functions are quite challenging because of a frequently changing behavior and the need to use a large number of data points for an accurate approximation. Tables 1–3 provide quantitative information about the quality of fit and the speed of evaluation.

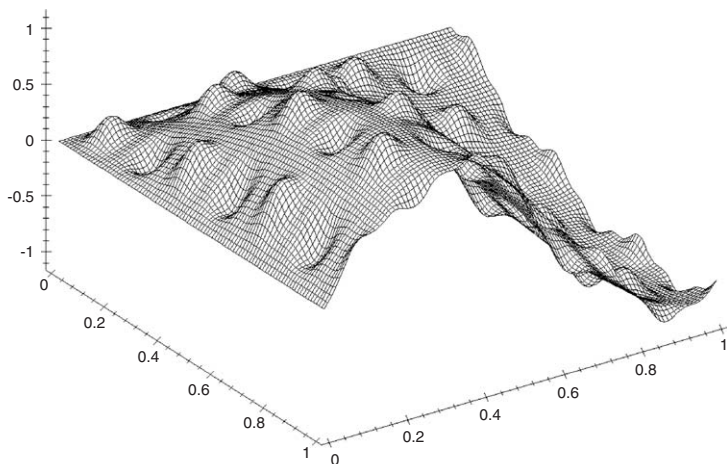


Fig. 5. Test function 1.

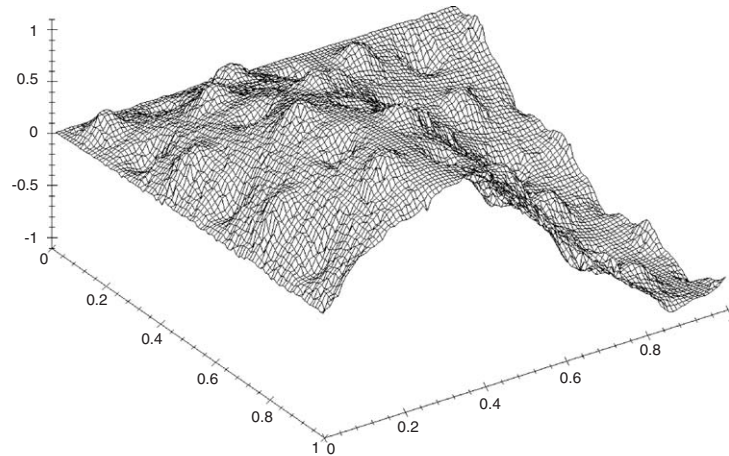


Fig. 6. Uniform approximation of the test function 1 using 20 000 data points.

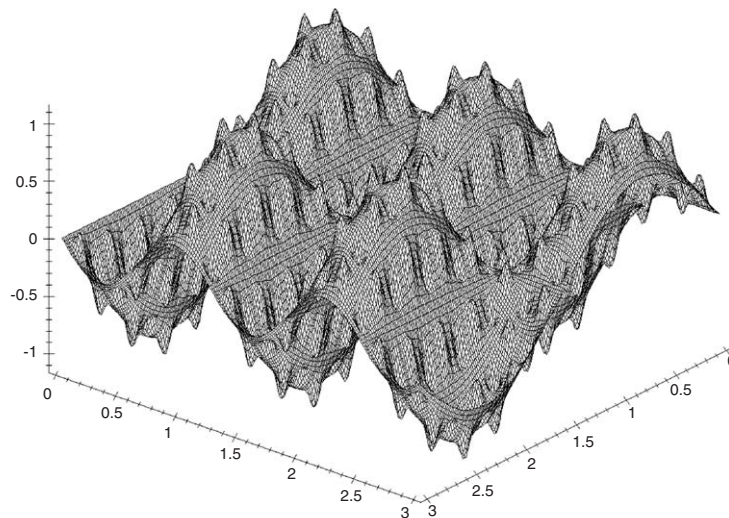


Fig. 7. Test function 2.

There are two steps of the algorithm that need benchmarking. The first step of building the interpolant g is called preprocessing, and the second step is the evaluation of g for an arbitrary x . Evaluation step was performed 100 000 times at random points to gather statistics, and the average time is reported. Further, the maximum and mean errors of approximation are reported. All computations were performed on Pentium-IV PC, 1.2 GHz, 512 MB Ram, Visual C++ (version 6) compiler.

To compare the performance to some existing methods, we present timing of preprocessing and evaluation steps of Sibson's natural neighbor scheme in 2 and 3 variables. We are unaware of an efficient natural neighbor interpolation algorithm for more than 3 variables. The results are presented in [Table 4](#),

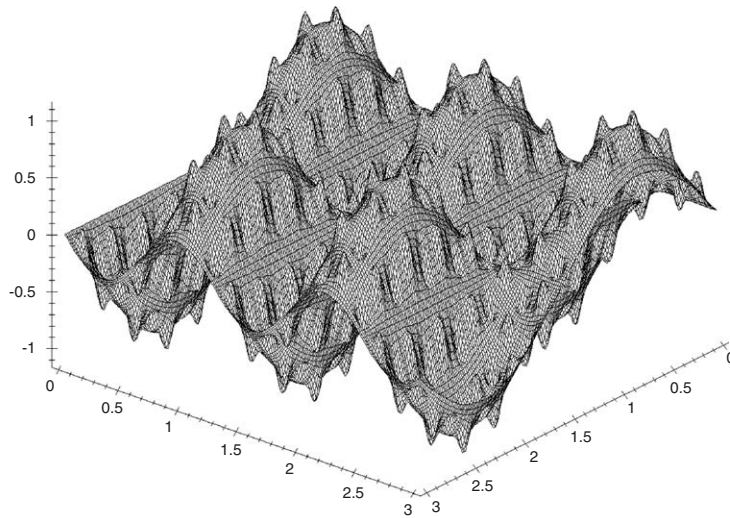


Fig. 8. Uniform approximation of the test function 2 using 80 000 data points.

Table 1

Performance of the algorithm for test function 1 as a function of the number of data points

K	Preprocessing times (s)	Evaluation time ($s \times 10^{-3}$)	Max error	Root mean squared error
10 000	1.02	0.29	0.17	0.025
20 000	2.39	0.28	0.12	0.018
40 000	5.58	0.39	0.092	0.013
80 000	12.88	0.41	0.069	0.0090

Table 2

Performance of the algorithm for test function 2 as a function of the number of data points

K	Preprocessing time (s)	Evaluation time ($s \times 10^{-3}$)	Max error	Root mean squared error
10 000	1.02	0.25	0.34	0.045
20 000	2.43	0.28	0.18	0.031
40 000	5.83	0.34	0.15	0.021
80 000	12.90	0.40	0.021	0.013

An implementation of the natural neighbor algorithm in 2 dimensions called `nnbathy` was used for testing purposes. It is available from www.marine.csiro.au/~sakov/. The natural neighbor algorithm in 3 dimensions was provided by M. Sambridge and J. Braun from the Australian National University, see www.rses.anu.edu.au/geodynamics/nn/nn.html.

It is also interesting to compare the performance of the preprocessing step of our algorithm with the time needed to build the Voronoi diagram (or its dual Delaunay triangulation). The reason is that both nearest neighbor methods and triangulation based schemata include it as an essential preprocessing step.

Table 3

Performance of the algorithm for test function 3 as a function of the number of data points and dimension

m	K	Preprocessing time (s)	Evaluation time ($s \times 10^{-3}$)	Exhaustive evaluation ($s \times 10^{-3}$)	Max error	Root mean squared error
2	1000	0.07	0.14	0.32	0.35	0.025
	10 000	1.02	0.22	3.2	0.34	0.020
	20 000	2.39	0.26	6.3	0.12	0.016
	40 000	5.58	0.29	13.0	0.091	0.013
	80 000	12.9	0.31	26.1	0.051	0.0090
3	1000	0.17	0.72	0.38	0.63	0.14
	10 000	2.81	1.20	3.6	0.32	0.063
	20 000	6.67	1.46	7.1	0.27	0.050
	40 000	15.69	1.55	14.3	0.18	0.038
	80 000	35.57	1.61	27.8	0.17	0.031
4	1000	0.78	2.57	0.44	1.01	0.19
	5000	7.29	5.48	2.04	0.72	0.13
	10 000	18.2	7.06	3.9	0.61	0.11
	20 000	45.3	8.92	8.1	0.33	0.080
	40 000	110.0	12.2	16.1	0.29	0.076
5	1000	4.66	15.8	0.48	1.04	0.19
	5000	54.08	44.1	2.7	0.83	0.14
	10 000	149.7	62.5	5.3	0.74	0.13

Exhaustive evaluation refers to directly using Eq. (9) in computations, whose complexity grows linearly with K .

Table 4

Performance of the Sibson's natural neighbor interpolant (test function 3) as a function of the number of data points and dimension

m	K	Preprocessing time (s)	Evaluation time ($s \times 10^{-3}$)
2	10 000	0.91	0.11
	20 000	1.51	0.24
	40 000	5.31	0.31
	80 000	10.1	0.44
3	10 000	3.16	2.63
	20 000	6.48	2.67
	40 000	13.31	2.75
	80 000	19.96	3.10

Thus the time reported in Table 5 is a lower bound for the preprocessing step of both categories of methods. We used `qhull` software for computations, available from www.qhull.org.

Table 3 confirms exponential dependence of the computational complexity of the preprocessing algorithm on the dimension m . At the end of Section 4.3 we mentioned that the complexity of the evaluation step is $O(\log K)$ when using our algorithm, and is $O(K)$ when using exhaustive evaluation by Eq. (9). Table 3 also confirms this.

Table 5

Performance of the `qhull` algorithm building the Voronoi diagram/Delaunay triangulation of a set of sites as a lower bound on preprocessing time of the natural neighbor and triangulation based interpolants

m	K	Time for building Voronoi diagram (s)
2	10 000	< 1
	20 000	1.5
	40 000	5.0
	80 000	10.0
3	10 000	2.2
	20 000	4.2
	40 000	8.7
	80 000	18.6
4	1000	2.1
	5000	8.2
	10 000	17.4
	20 000	37.4
5	40 000	82.0
	1000	13.8
	5000	97.2
	10 000	211.0

By comparing [Tables 3](#) and [4](#), one can see that the proposed method is competitive with the natural neighbor schemata for $m = 2, 3$, both in terms of preprocessing and evaluation time. Further, [Table 5](#) reveals that the preprocessing time of our method is of the same magnitude as the lower bound on preprocessing time of the alternative methods for bigger m .

As m increases, the construction of the upper and lower approximations becomes very challenging, as the number of local optima of these functions grows very fast, and memory limitations of current computers do not allow one to efficiently process them in RAM. The same applies to building Voronoi diagrams/Delaunay triangulations, the reason why we did not extend [Tables 3–5](#), for larger values of m and K . However, [Table 3](#) reveals that for some combinations of values m and K , direct evaluation of (9) may be more efficient than using saw-tooth underestimate construction in Section 4. Thus, the applicability of our method is not constrained by the complexity of the Voronoi diagram. Unlike the alternative natural neighbor and Delaunay triangulation based methods, the proposed interpolant can be evaluated with zero preprocessing time, and still acceptable evaluation time.

7. Conclusion

We presented a new method of multivariate scattered data interpolation which preserves Lipschitz properties of a function. A distinctive feature of this method is that it produces the optimal interpolant, whose accuracy is no worse as the intrinsic error of the interpolation problem. It is based on a well-known central interpolation scheme which approximates the unknown function best in the worst case scenario, and thus delivers reliability of the approximation process, and best uniform error bounds. In addition, the

interpolant possesses a number of desirable properties, such as locality of the approximation scheme and continuous dependence on the data.

The main contribution of this paper is the development of a particularly fast algorithm for computation of the interpolant in more than one variable. It involves an alternative representation of the interpolant using the list of its local minima and maxima, and is equivalent to building an additively weighted Voronoi diagram. With the help of a special simplicial distance, we developed an efficient computational algorithm, competitive with those used to build ordinary Voronoi diagrams—which is a much simpler problem.

The new method favorably compares with the alternative methods for this class of problems. It provides short computation time (logarithmic in the number of data points) and a competitive preprocessing time. It can also be used with no preprocessing, but with longer evaluation time. This feature gives the user unique flexibility as to which evaluation method to use. The computation of the interpolant is straightforward and numerically stable, it does not require solving linear systems of equations. Theoretical guarantees on the accuracy of approximation are tight.

The new method can be used for reliable approximation of functions of more than three variables using a large number of data points (tens of thousands or more), on an ordinary PC.

References

- [1] P. Alfeld, Scattered data interpolation in three or more variables, in: L.L. Schumaker, T. Lyche (Eds.), *Mathematical Methods in Computer Aided Geometric Design*, Academic Press, New York, 1989, pp. 1–34.
- [2] F. Aurenhammer, Voronoi diagrams—a survey of a fundamental data structure, *ACM Computing Surveys* 23 (1991) 345–405.
- [3] N.S. Bakhvalov, On the optimality of linear methods for operator approximation in convex classes of functions, *U.S.S.R. Comput. Math. and Math. Phys.* 11 (1971) 244–249.
- [4] L.M. Batten, G. Beliaikov, Fast algorithm for the cutting angle method of global optimization, *J. Global Optim.* 24 (2002) 149–161.
- [5] G. Beliaikov, Shape preserving approximation using least squares splines, *Approx. Theory Appl.* 16 (2000) 80–98.
- [6] G. Beliaikov, Geometry and combinatorics of the cutting angle method, *Optimization* 52 (2003) 379–394.
- [7] G. Beliaikov, How to build aggregation operators from data, *Internat. J. Intelligent Systems* 18 (2003) 903–923.
- [8] G. Beliaikov, Extended cutting angle method of constrained global optimization, in: L. Caccetta (Ed.), *Optimization in Industry*, Springer, Heidelberg, 2005, in press.
- [9] V.V. Belikov, V.D. Ivanov, V.K. Kontorovich, S.A. Korytnik, A.Y. Semenov, The non-sibsonian interpolation: a new method of interpolation of the values of a function on an arbitrary set of points, *Comput. Math. and Math. Phys.* 37 (1997) 9–15.
- [10] J.-D. Boissonnat, M. Sharir, B. Tagansky, M. Yvinec, Voronoi diagrams in higher dimensions under certain polyhedral distance functions, *Discrete Comput. Geom.* 19 (1998) 485–519.
- [11] V. Cherkassky, F. Mulier, *Learning from Data*, Wiley, New York, 1998.
- [12] D.A. Cooper, Learning Lipschitz functions, *Internat. J. Comput. Math.* 59 (1995) 15–26.
- [13] C. De Boor, Computational aspects of optimal recovery, in: C.A. Micchelli, T.J. Rivlin (Eds.), *Optimal Estimation in Approximation Theory*, Plenum Press, New York, 1977, pp. 69–91.
- [14] V.F. Demyanov, A.M. Rubinov, *Constructive Nonsmooth Analysis*, Peter Lang, Frankfurt am Main, 1995.
- [15] L. Devroye, *Non-uniform Random Variate Generation*, Springer, New York, 1986.
- [16] P. Dierckx, *Curve and Surface Fitting with Splines*, Clarendon press, Oxford, 1995.
- [17] P.W. Gaffney, M.J.D. Powell, Optimal interpolation, in: G.A. Watson (Ed.), *On Numerical Analysis, Lecture Notes in Mathematics*, vol. 506, Springer, Heidelberg, 1976, pp. 90–99.
- [18] M. Golomb, H.F. Weinberger, Optimal approximation and error bounds, in: R.E. Langer (Ed.), *On Numerical Approximation*, The University of Wisconsin Press, Madison, 1959, pp. 117–190.
- [19] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, Springer, New York, Berlin, Heidelberg, 2001.
- [20] L.M. Kocic, G.V. Milovanovic, Shape-preserving approximations by polynomials and splines, *Comput. Math. Appl.* 3 (1997) 59–97.

- [21] B. Kvasov, *Methods of Shape Preserving Spline Approximation*, World Scientific, Singapore, 2000.
- [22] S.R. Lay, *Convex Sets and their Applications*, Wiley, New York, 1972.
- [23] C.A. Micchelli, T.J. Rivlin, A survey of optimal recovery, in: C.A. Micchelli, T.J. Rivlin (Eds.), *Optimal Estimation in Approximation*, Plenum Press, New York, 1977, pp. 1–54.
- [24] C.A. Micchelli, T.J. Rivlin, Lectures on optimal recovery, in: P.R. Turner (Ed.), *Numerical Analysis, Lecture Notes in Mathematics*, vol. 1129, Springer, Heidelberg, 1984, pp. 12–93.
- [25] C.A. Micchelli, T.J. Rivlin, S. Winograd, The optimal recovery of smooth functions, *Numer. Math.* 26 (1976) 191–200.
- [26] A. Okabe, B. Boots, K. Sugihara, S.N. Chiu, *Spatial Tessellations: Concepts and Applications of Voronoi Diagrams*, second ed., Wiley, Chichester, 2000.
- [27] S.A. Pijavski, An algorithm for finding the absolute extremum of a function, *U.S.S.R. Comput. Math. and Math. Phys.* 2 (1972) 57–67.
- [28] A.M. Rubinov, *Abstract Convexity and Global Optimization, Nonconvex Optimization and its Applications*, vol. 44, Kluwer Academic Publishers, Dordrecht, Boston, 2000.
- [29] B. Shubert, A sequential method seeking the global maximum of a function, *SIAM J. Numer. Anal.* 9 (1972) 379–388.
- [30] R. Sibson, A brief description of natural neighbor interpolation, in: V. Barnett (Ed.), *Interpreting Multivariate Data*, Wiley, Chichester, 1981, pp. 21–36.
- [31] K. Sikorski, *Optimal Solution of Nonlinear Equations*, Oxford University Press, Oxford, 2001.
- [32] S.A. Smolyak, Quadrature and interpolation formulas for tensor products of certain classes of functions, *Math. U.S.S.R. Doklady* 4 (1963) 240–243.
- [33] R.G. Strongin, Y.D. Sergeyev, *Global Optimization with Non-convex Constraints: Sequential and Parallel Algorithms, Nonconvex Optimization and its Applications*, vol. 45, Kluwer Academic, Dordrecht, London, 2000.
- [34] A.G. Sukharev, Optimal search for the roots of a function satisfying a Lipschitz condition, *U.S.S.R. Comput. Math. and Math. Phys.* 16 (1976) 17–26.
- [35] A.G. Sukharev, Optimal method of constructing best uniform approximation for functions of a certain class, *U.S.S.R. Comput. Math. and Math. Phys.* 18 (1) (1978) 21–31.
- [36] A.G. Sukharev, On the existence of optimal affine methods for approximating linear functionals, *J. Complexity* 2 (1986) 317–322.
- [37] A.G. Sukharev, *Minimax Models in the Theory of Numerical Methods, Theory and Decision Library Series B*, vol. 21, Kluwer Academic Publishers, Dordrecht, 1992.
- [38] A.H. Thiessen, Precipitation averages for large areas, *Monthly Weather Report* 39 (1911) 1082–1084.
- [39] J.F. Traub, H. Wozniakowski, *A General Theory of Optimal Algorithms*, Academic Press, New York, 1980.
- [40] G.R. Wood, B.P. Zhang, Estimation of the Lipschitz constant of a function, *J. Global Optim.* 8 (1996) 91–103.
- [41] Z.B. Zabinsky, B.P. Kristinsdottir, R.L. Smith, Optimal estimation of univariate black box Lipschitz functions with upper and lower error bounds, *Internat. J. Comput. Res.* 30 (2003) 1539–1553.